JIALONG MEI

Backend & Applied AI Engineer · Amsterdam, NL

JialongM@outlook.com linkedin.com/in/jialong-mei-38b93a27b github.com/JialongMei

EXPERIENCE

Full Stack Engineer · legal-tech company

Apr 2025 – present · Amsterdam

Backend Developer Intern · Zhejiang Topthinking Information Technology

Mar 2024 – Aug 2024 · Zhejiang, China

SELECTED WORK

① LegalTech platform

Production platform for Dutch and Belgian legal cases. I work on every layer of it, through to the Kubernetes releases.

EVIDENCE Replaced per-thread boto3 sessions with a locked process-level cache. File-URL generation went from about 240 ms to 1 ms per request on staging.

- Added PostgreSQL indexes concurrently on a production table with millions of rows, so deploys did not hold long write locks.

② OCR → deadline pipeline

Legal mail arrives as large scanned batches. The pipeline splits each batch into letters, routes every letter to a human-approved template, extracts the deadline and other fields with their evidence, and sends anything uncertain to a person at a review station.

- A self-hosted vision model on our own GPU parses each page into layout blocks. The image is pinned by digest, page artifacts are stored encrypted, and the GPU stops when idle.
- A deterministic splitter cuts a batch into letters by anchors and case identifiers and keeps consecutive pages of the same case together.
- Routing uses only fixed template versions that a person has approved. A missing anchor or an ambiguous match sends the letter to manual review.
- Every extracted field links to its evidence on the page, and validators such as reference-number format checks decide whether automation may continue.
- Staff work in one review station with three workspaces: the daily review queue, template evidence, and a view of how each batch was split. Their corrections become proposed rules that a human reviewer approves.

③ Legal AI assistants

LLM assistants that legal staff use inside the platform, including a live chat.

- Tools return typed, structured output.
- Legal-domain embeddings find similar cases and add them as context. Attachments can be images or PDFs.
- Cost is controlled with prompt caching, usage tracking and rate limits.

④ Agent fleet

How I develop: a planner agent writes a spec, a worker agent implements it in its own git worktree, and a separate reviewer agent accepts or rejects the result.

- The spec has acceptance criteria before any code is written.
- Workers do not push. Each change is reviewed on its own; verified branches merge into one integration branch per batch, which gets one test run and one pull request.
- Hayabusa is my fork of an open-source (MIT) agent distro. I added the kamae supervisor CLI, the shared approval guard and the tool-capability contract.
- The reviewer reads the diff and the test output. It does not see the worker's summary.

⑥ EBEP energy study

MSc thesis comparing the energy use of two CI/CD setups: one creates a fresh preview environment for each change, the other keeps environments running.

EVIDENCE Across 44 runs, ephemeral environments used 60–67 % less energy over 24 hours, with no measured loss in service quality.

- Replayed the same k6 workload at 1, 25 and 50 sessions per workday, on AWS and on bare-metal Kubernetes.
- Measured energy with calibrated CCF estimates and Kepler-RAPL, with resumable runs and per-container data checks.
- The harness is MIT-licensed and comes with four CSV datasets, about 85,600 rows.

⑤ Quant research

Personal research platform for testing equity factors on point-in-time data.

- Data pipelines for SEC EDGAR fundamentals, long-horizon prices and analyst estimates, each stored as it was known on its date.
- Combinatorial purged cross-validation, a protocol frozen before each run, and synthetic tests for temporal leakage.

EDUCATION

MSc Computer Science · University of Amsterdam & Vrije Universiteit Amsterdam

Sep 2024 – Aug 2026

Thesis, graded 9/10: energy trade-offs between ephemeral and persistent preproduction environments in cloud-native CI/CD. github.com/JialongMei/ebep-harness

BSc Computer Science · Vrije Universiteit Amsterdam

Sep 2020 – Jul 2023

SKILLS

Languages
Python, TypeScript, JavaScript, SQL, Java, Bash

Backend and data
Django, Django REST Framework, Celery, Redis, PostgreSQL, pandas

Frontend
React, HTML, CSS

Cloud and DevOps
Docker, Kubernetes, Rancher, AWS (RDS, ECR, S3), Bitbucket Pipelines

Applied AI
Structured tool use, multimodal documents, retrieval and embeddings, prompt versioning, token and latency tracking

Spoken
Chinese (native), English (full professional)

OFF-DUTY

Outside work I hike, mostly in the mountains.